# Buenas prácticas de API REST

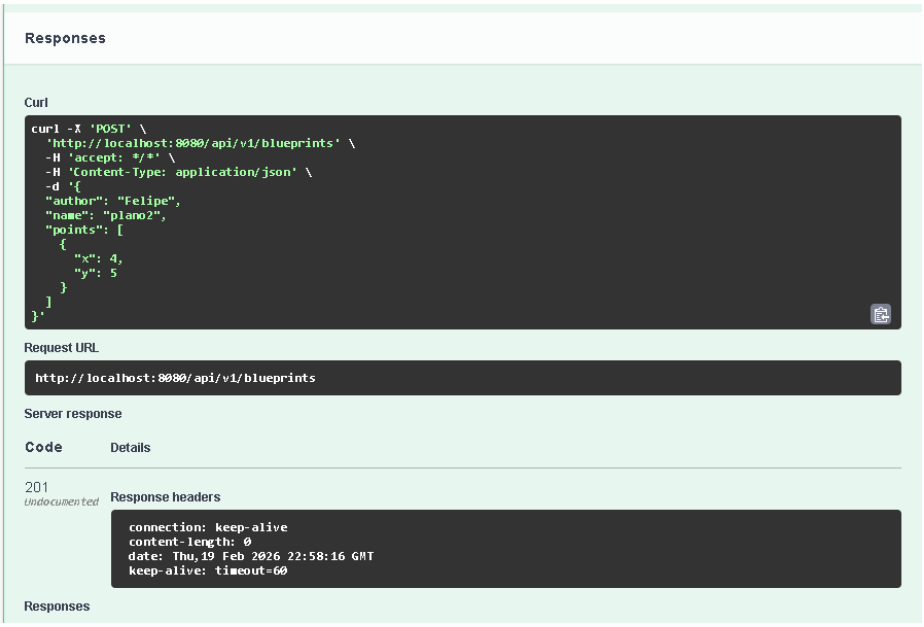
Primero vamos a cambiar el path base de / blueprints a la ruta /api/v1/blueprints.

**Objetivo**

- Permitir evolución futura sin romper clientes existentes.
- Facilitar mantenimiento y control de versiones.
- Adoptar estándar REST moderno.

**Prueba HTTP**

**Mensaje 201**




El mensaje 201 hace referencia a un recurso creado correctamente

**Mensaje 200**

```
curl -X 'GET' \
  'http://localhost:8080/api/v1/blueprints' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/v1/blueprints
```

Server response

| Code | Details |
|---|---|
| 200 | Response body |


```
[
  {
    "author": "Felipe",
    "name": "plano2",
    "points": [
      {
        "x": 4,
        "y": 5
      }
    ]
  },
  {
    "author": "oscar",
    "name": "plano1",
    "points": [
      {
        "x": 1,
        "y": 2
      },
      {
        "x": 3,
        "y": 4
      }
    ]
  }
]
```


Download

Hace referencia a una consulta correcta

**Mensaje 202**

Curl

```
curl -X 'PUT' \
  'http://localhost:8080/api/v1/blueprints/Felipe/plano2/points' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "x": 1,
  "y": 1
}'
```

Request URL

```
http://localhost:8080/api/v1/blueprints/Felipe/plano2/points
```

Server response

| Code | Details |
|---|---|
| 202 *Undocumented* | Response headers |

```
connection: keep-alive
content-length: 0
date: Thu,19 Feb 2026 23:03:00 GMT
keep-alive: timeout=60
```

Responses

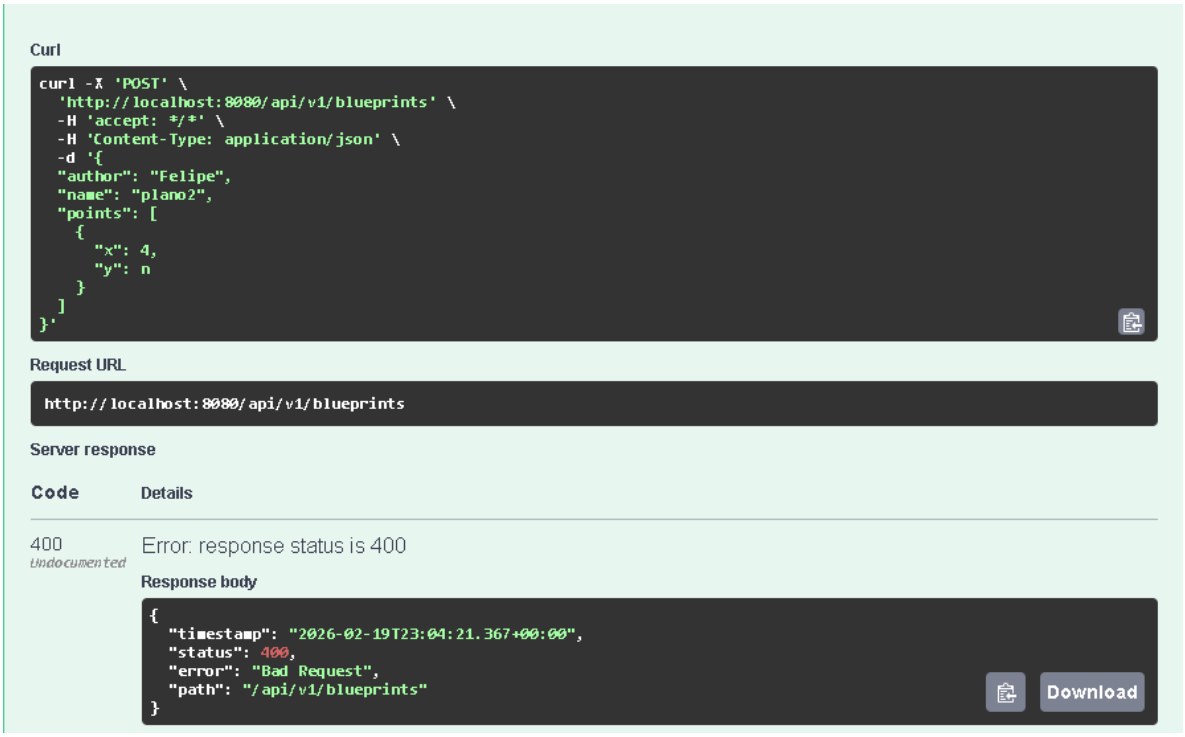
| Code | Description | Links |
|---|---|---|
| 200 | OK | *No links* |

Modificación aceptada

**Mensaje 404**


Recurso no encontrado

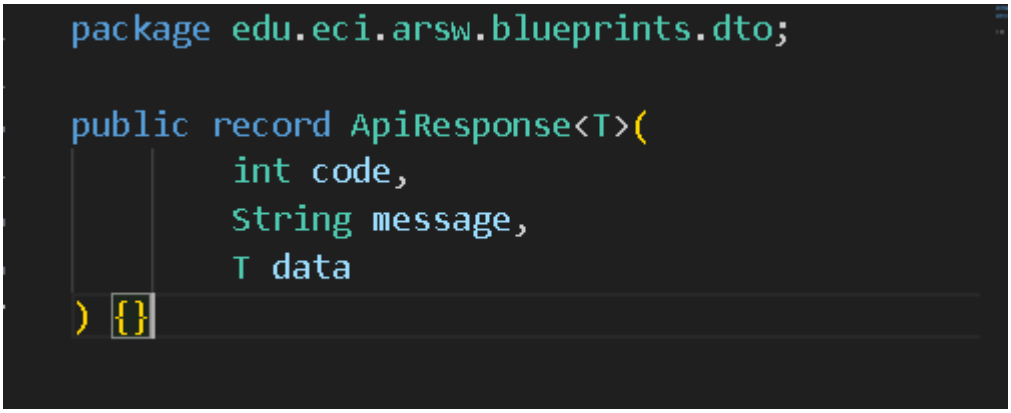
**Mensaje 400**




Datos erróneos

**Crear respuesta uniforme ApiResponse<T>**

Se implementó una estructura uniforme para todas las respuestas




**Beneficios**

- Respuestas consistentes.
- Manejo uniforme de errores.
- Mayor claridad semántica.

Luego Se ajustaron los códigos de respuesta según la operación

```java
// GET /blueprints
@GetMapping
public ResponseEntity<ApiResponse<Set<Blueprint>>> getAll() {
    Set<Blueprint> data = services.getAllBlueprints();
    return ResponseEntity.ok(
            new ApiResponse<>(200, "execute ok", data));
}

// GET /blueprints/{author}
@GetMapping("/{author}")
public ResponseEntity<?> byAuthor(@PathVariable String author) {
    try {
        Set<Blueprint> data = services.getBlueprintsByAuthor(author);
        return ResponseEntity.ok(
                new ApiResponse<>(200, "execute ok", data));
    } catch (BlueprintNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(new ApiResponse<>(404, e.getMessage(), null));
    }
}
```

```java
// GET /blueprints/{author}/{bpname}
@GetMapping("/{author}/{bpname}")
public ResponseEntity<?> byAuthorAndName(@PathVariable String autho

    try {
        Blueprint data = services.getBlueprint(author, bpname);
        return ResponseEntity.ok(
                new ApiResponse<>(200, "execute ok", data));
    } catch (BlueprintNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(new ApiResponse<>(404, e.getMessage(), null));
    }
}

// POST /blueprints
@PostMapping
public ResponseEntity<ApiResponse<?>> add(@Valid @RequestBody NewBl
    try {
        Blueprint bp = new Blueprint(req.author(), req.name(), req.
        services.addNewBlueprint(bp);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(new ApiResponse<>(201, "Blueprint created", nu
    } catch (BlueprintPersistenceException e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
                .body(new ApiResponse<>(403, e.getMessage(), null));
    }
}
```

```java
    // PUT /blueprints/{author}/{bpname}/points
    @PutMapping("/{author}/{bpname}/points")
    public ResponseEntity<?> addPoint(@PathVariable String author, @Path
            @RequestBody Point p) {
        try {
            services.addPoint(author, bpname, p.x(), p.y());
            return ResponseEntity.status(HttpStatus.ACCEPTED)
                    .body(new ApiResponse<>(202, "Point added", null));
        } catch (BlueprintNotFoundException e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                    .body(new ApiResponse<>(404, e.getMessage(), null));
        }
    }

    public record NewBlueprintRequest(
            @NotBlank String author,
            @NotBlank String name,
            @Valid java.util.List<Point> points) {
    }
}
```

Se implementó un @RestControllerAdvice para capturar errores de validación:

```java
ckage edu.eci.arsw.blueprints.exceptions;

port edu.eci.arsw.blueprints.dto.ApiResponse;
port org.springframework.http.HttpStatus;
port org.springframework.web.bind.MethodArgumentNotValidException;
port org.springframework.web.bind.annotation.*;

estControllerAdvice
lic class GlobalExceptionHandler {

  @ExceptionHandler(MethodArgumentNotValidException.class)
  @ResponseStatus(HttpStatus.BAD_REQUEST)
  public ApiResponse<?> handleValidation(MethodArgumentNotValidExceptio
      return new ApiResponse<>(400, "Invalid request data", null);
  }
```

Esto evita respuestas HTML por defecto y mantiene el estándar JSON.